Unit 5: Exception Handling

Introduction to Exception

A Java exception is an object that represents an error or an unexpected condition that occurs during program execution.

When such a condition arises, an exception object is created and thrown by the method where the error occurred.

That method can either handle the exception itself or pass it on to the caller.

Eventually, the exception is caught and processed to prevent the program from crashing.

This mechanism helps handle runtime errors and allows the program to continue running smoothly.

Terminologies

Try: The try block encloses code that might throw an exception. Think of it as a **safety net** ready to catch any unexpected errors.

Catch: The catch block handles exceptions thrown by the try block. It acts like a **rescue team** that takes care of the problem.

Throw: The throw keyword is used to explicitly throw an exception. It's like **raising an alarm** to signal that something has gone wrong.

Throws: The throws keyword declares that a method **might throw** an exception. It's a **warning sign** telling callers to be prepared for potential errors.

Finally: The finally block executes code **no matter what**, whether an exception occurred or not. It's the **cleanup crew** that tidies up after everything.

```java
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            // Code that might throw an exception
            System.out.println("Trying to divide by zero...");
            int result = divide(10, 0);
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            // Handle the exception
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } finally {
            // Execute code regardless of whether an exception was thrown
            System.out.println("Finally, I'm done!");
        }
    }

    public static int divide(int a, int b) throws ArithmeticException {
        // Throw an exception if b is zero
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero!");
        }
        return a / b;
    }
}
```

Errors in Java Program

An Error in Java represents a serious problem that usually stops the normal execution of a program.

It is a subclass of Throwable and indicates a condition that a reasonable application should not try to catch or recover from.

Errors and their subclasses are part of unchecked exceptions, but they are different from regular exceptions because they are usually beyond the control of the program.

Errors cannot typically be handled or solved by the program's logic. Instead, they indicate serious issues in the runtime environment.

Common examples of errors in Java include OutOfMemoryError and StackOverflowError.

References: https://www.javatpoint.com/java-error

Exceptions

Exceptions are problems that occur during the execution of a program.

They represent conditions that a well-designed program can anticipate and handle, such as FileNotFoundException or ArrayIndexOutOfBoundsException.

An exception is an object that is thrown when an error occurs at runtime.

Exceptions can and should be handled using try-catch blocks to allow the program to recover gracefully and continue running.

References: https://www.javatpoint.com/exception-handling-in-java

Types of Exceptions

1. Checked Exception
2. Unchecked Exception

References:
https://www.geeksforgeeks.org/checked-vs-unchecked-exceptions-in-java/

https://sawin.com.np

Checked Exceptions

1. Checked exceptions are exceptions that the compiler checks at compile time.
2. They must be either handled with try-catch blocks or declared in the method signature using throws.
3. Checked exceptions are subclasses of Exception, but not subclasses of RuntimeException.
4. Common examples include IOException, SQLException, and ClassNotFoundException.

Unchecked Exception

1. Runtime exceptions are exceptions that the compiler does not check during compilation.
2. The programmer is not required to handle or declare them in the method signature.
3. They are subclasses of RuntimeException, which itself is a subclass of Exception.
4. Common examples include NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, and IllegalArgumentException.

Built in exceptions in Java

Java provides several built-in exception classes in the java.lang package.

Many of these are unchecked exceptions, which are subclasses of RuntimeException.

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

TABLE 10-1 Java's Unchecked **RuntimeException** Subclasses Defined in **java.lang**

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

TABLE 10-2 Java's Checked Exceptions Defined in **java.lang**

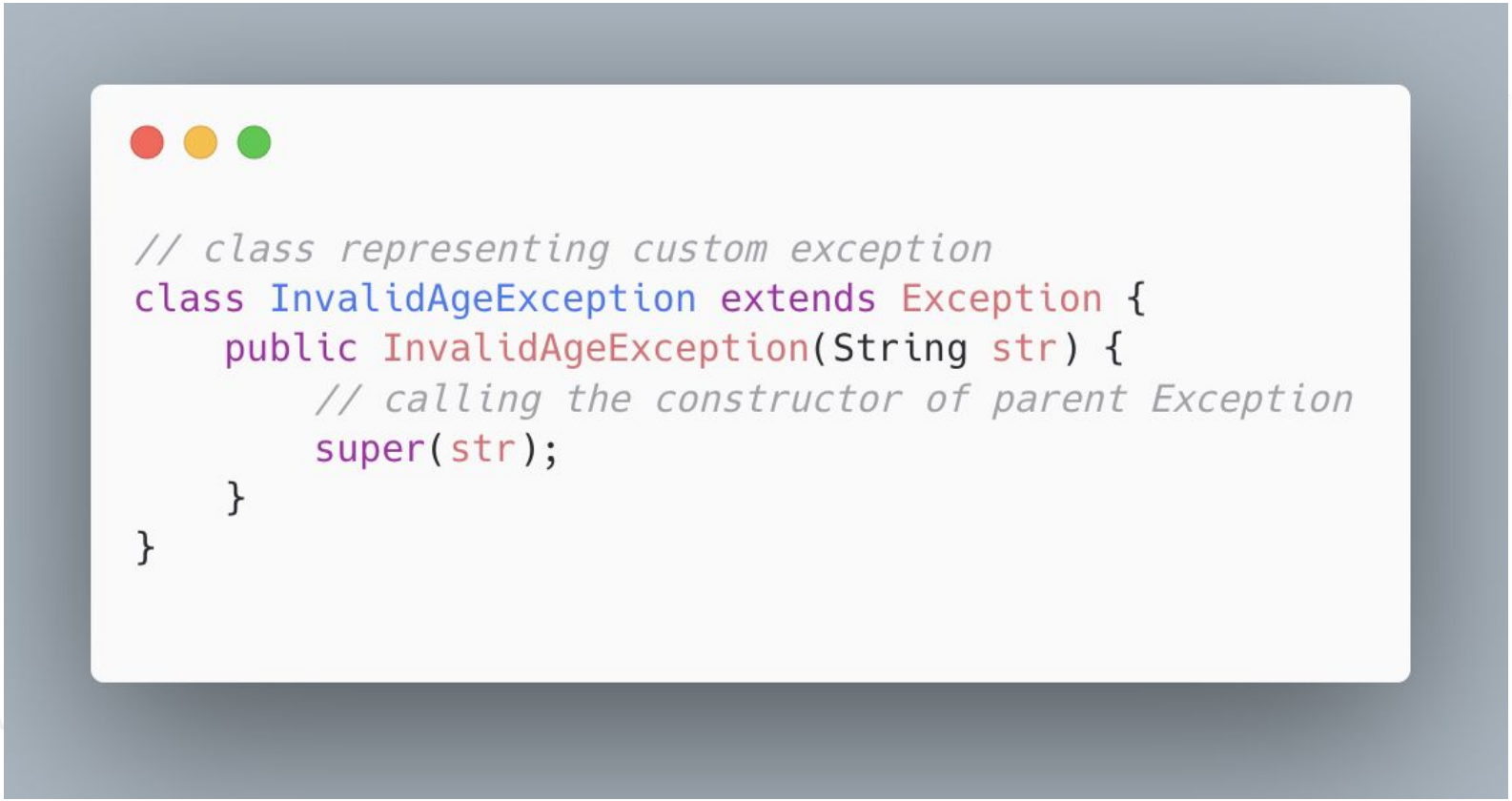
User defined exception in Java

In Java, we can create our own exceptions by extending the Exception class.

These are called custom exceptions or user-defined exceptions.

Custom exceptions allow you to define your own error conditions to suit the specific needs of your application.

Example



```
// class representing custom exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String str) {
        // calling the constructor of parent Exception
        super(str);
    }
}
```

```java
// class that uses custom exception InvalidAgeException
public class CustomException {

    // method to check the age
    static void validate(int age) throws InvalidAgeException {
        if (age < 18) {

            // throw an object of user defined exception
            throw new InvalidAgeException("age is not valid to vote");
        } else {
            System.out.println("welcome to vote");
        }
    }

    // main method
    public static void main(String args[]) {
        try {
            // calling the method
            validate(19);
        } catch (InvalidAgeException ex) {
            System.out.println("Caught the exception");

            // printing the message from InvalidAgeException object
            System.out.println("Exception occured: " + ex);
        }

        System.out.println("rest of the code...");
    }
}
```

Practical

1. Write a Java program that demonstrates the basic concept of an exception. Create an array of size 5 and try to access the 10th element. Catch the exception and print an appropriate message.
2. Write a Java program that intentionally causes a NullPointerException. Catch the exception and print a message indicating the error.
3. Write a Java program that demonstrates the difference between a checked and an unchecked exception. Use FileNotFoundException for the checked exception and ArithmeticException for the unchecked exception.
4. Write a Java program that demonstrates the use of try, catch, finally, and throw keywords. Create a method that throws an IllegalArgumentException if a negative number is passed to it.
5. Write a Java program that handles multiple built-in exceptions. Use NumberFormatException and ArrayIndexOutOfBoundsException in your program.
6. Write a Java program that creates a user-defined exception called InvalidAgeException. This exception should be thrown if the age provided is less than 18. Create a method that validates the age and throws this exception if the condition is met.